

MODULE – 5

TRANSACTION PROCESSING

INTRODUCTION TO TRANSACTION PROCESSING

CONCURRENCY CONTROL IN DATABASES

**INTRODUCTION TO DATABASE RECOVERY
PROTOCOLS**

Introduction to Transaction Processing

Transaction: An executing program (process) that includes one or more database access operations

Read operations (database retrieval, such as SQL SELECT)

Write operations (modify database, such as SQL INSERT, UPDATE, DELETE)

Transaction: A logical unit of database processing

Example: Bank balance transfer of \$100 dollars from a checking account to a saving account in a BANK database

Note: Each execution of a program is a distinct transaction with different parameters

Bank transfer program parameters: savings account number, checking account number, transfer amount

Introduction to Transaction Processing (cont.)

- **A transaction (set of operations)** may be:
 - stand-alone, specified in a high level language like SQL submitted interactively, or
 - consist of database operations embedded within a program (most transactions)
- **Transaction boundaries:** Begin and End transaction.
 - Note: An **application program** may contain several transactions separated by Begin and End transaction boundaries

Introduction to Transaction Processing (cont.)

- **Transaction Processing Systems:** Large multi-user database systems supporting thousands of *concurrent transactions* (user processes) per minute
- **Two Modes of Concurrency**
 - **Interleaved processing:** concurrent execution of processes is interleaved in a single CPU
 - **Parallel processing:** processes are concurrently executed in multiple CPUs (Figure)
 - Basic transaction processing theory assumes interleaved concurrency

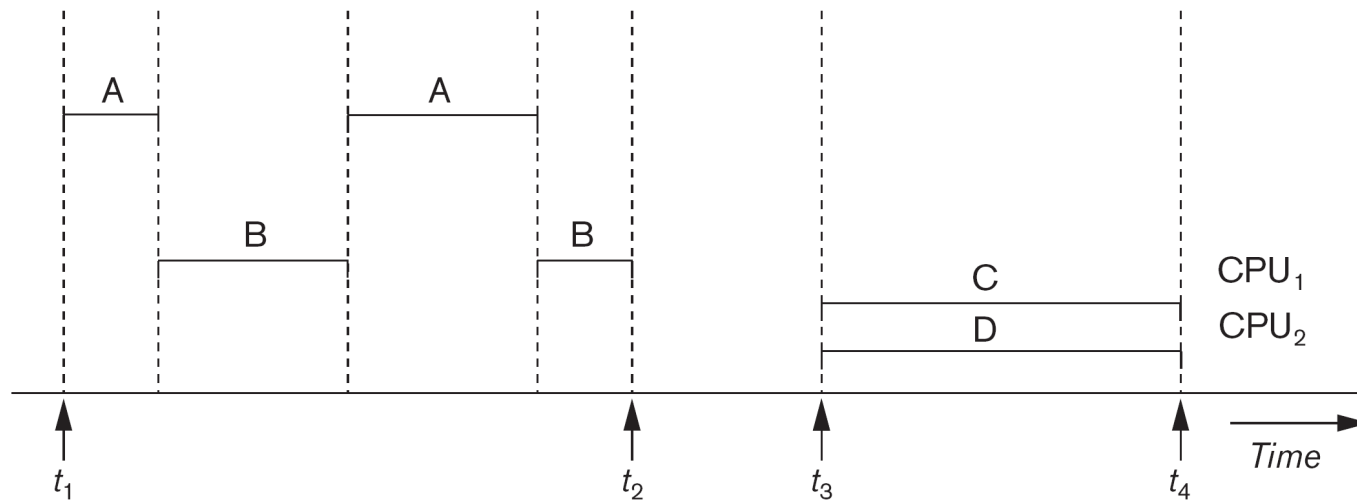


Figure 21.1
Interleaved processing versus parallel processing of concurrent transactions.

Introduction to Transaction Processing (cont.)

For transaction processing purposes, a simple database model is used:

- **A database** - collection of named data items
- **Granularity (size) of a data item** - a field (data item value), a record, or a whole disk block
- TP concepts are independent of granularity
- **Basic operations on an item X:**
 - read_item(X):** Reads a database item named X into a program variable. To simplify our notation, we assume that *the program variable is also named X*.
 - write_item(X):** Writes the value of program variable X into the database item named X.

Introduction to Transaction Processing (cont.)

READ AND WRITE OPERATIONS:

Basic unit of data transfer from the disk to the computer main memory is one disk block (or page). A data item X (what is read or written) will usually be the field of some record in the database, although it may be a larger unit such as a whole record or even a whole block.

read_item(X) command includes the following steps:

- Find the address of the disk block that contains item X.
- Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
- Copy item X from the buffer to the program variable named X.

Introduction to Transaction Processing (cont.)

READ AND WRITE OPERATIONS (cont.):

write_item(X) command includes the following steps:

- Find the address of the disk block that contains item X.
- Copy that disk block into a buffer in main memory (if it is not already in some main memory buffer).
- Copy item X from the program variable named X into its correct location in the buffer.
- Store the updated block from the buffer back to disk (either immediately or at some later point in time).

Transaction Notation

- Figure (next slide) shows two examples of transactions
- Notation focuses on the read and write operations
- Can also write in shorthand notation:
 - $T1: b1; r1(X); w1(X); r1(Y); w1(Y); e1;$
 - $T2: b2; r2(Y); w2(Y); e2;$
- b_i and e_i specify transaction boundaries (begin and end)
- i specifies a unique transaction identifier (Tid)

(a)

T_1
read_item(X); $X := X - N$; write_item(X); read_item(Y); $Y := Y + N$; write_item(Y);

(b)

T_2
read_item(X); $X := X + M$; write_item(X);

Figure 21.2

Two sample transactions. (a) Transaction T_1 . (b) Transaction T_2 .

Why we need concurrency control

Without Concurrency Control, problems may occur with concurrent transactions:

Lost Update Problem.

Occurs when two transactions update the same data item, but both read the same original value before update (Figure (a), next slide)

The Temporary Update (or Dirty Read) Problem.

This occurs when one transaction T1 updates a database item X, which is accessed (read) by another transaction T2; then T1 fails for some reason (Figure (b)); X was (read) by T2 before its value is changed back (rolled back or UNDONE) after T1 fails

(a)

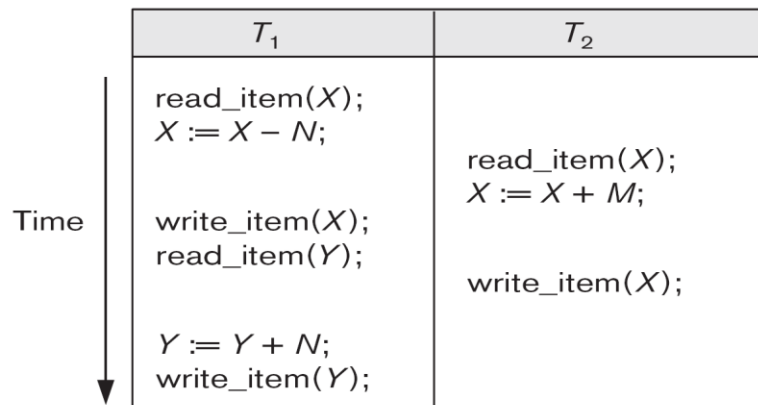
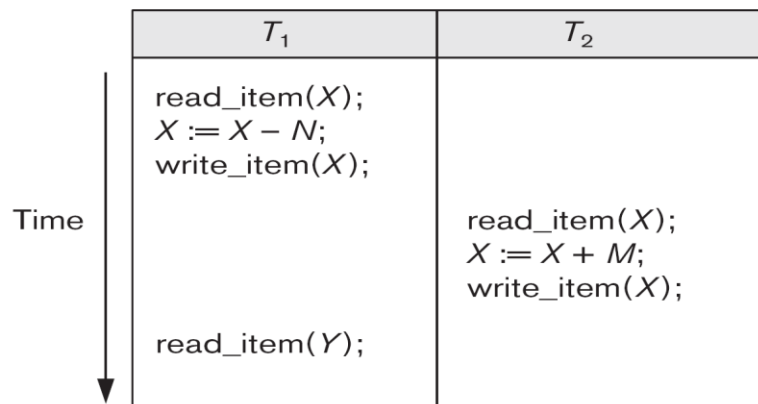


Figure 21.3

Some problems that occur when concurrent execution is uncontrolled. (a) The lost update problem. (b) The temporary update problem. (c) The incorrect summary problem.

← Item X has an incorrect value because its update by T_1 is *lost* (overwritten).

(b)



← Transaction T_1 fails and must change the value of X back to its old value; meanwhile T_2 has read the *temporary* incorrect value of X .

Why we need concurrency control (cont.)

The Incorrect Summary Problem

One transaction is calculating an aggregate summary function on a number of records (for example, sum (total) of all bank account balances) while other transactions are updating some of these records (for example, transferring a large amount between two accounts); the aggregate function may read some values before they are updated and others after they are updated.

(c)

T_1	T_3
<pre>read_item(X); X := X - N; write_item(X); read_item(Y); Y := Y + N; write_item(Y);</pre>	<pre>sum := 0; read_item(A); sum := sum + A; ⋮ read_item(X); sum := sum + X; read_item(Y); sum := sum + Y;</pre>

← T_3 reads X after N is subtracted and reads Y before N is added; a wrong summary is the result (off by N).

Why we need concurrency control (cont.)

- **The Unrepeatable Read Problem**

A transaction T1 may read an item (say, available seats on a flight); later, T1 may read the same item again and get a different value because another transaction T2 has updated the item (reserved seats on the flight) between the two reads by T1

Why recovery is needed

Causes of transaction failure:

1. **A computer failure (system crash):** A hardware or software error occurs during transaction execution. If the hardware crashes, the contents of the computer's internal main memory may be lost.
2. **A transaction or system error :** Some operation in the transaction may cause it to fail, such as integer overflow or division by zero. Transaction failure may also occur because of erroneous parameter values or because of a logical programming error. In addition, the user may interrupt the transaction during its execution.

Why recovery is needed (cont.)

3. Local errors or exception conditions detected by the transaction:

certain conditions necessitate cancellation of the transaction. For example, data for the transaction may not be found. A condition, such as insufficient account balance in a banking database, may cause a transaction, such as a fund withdrawal, to be canceled - a programmed abort causes the transaction to fail.

4. Concurrency control enforcement: The concurrency control method may decide to abort the transaction, to be restarted later, because it violates serializability or because several transactions are in a state of deadlock (see Chapter 22).

Why recovery is needed (cont.)

5. Disk failure: Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash. This kind of failure and item 6 are more severe than items 1 through 4.

6. Physical problems and catastrophes: This refers to an endless list of problems that includes power or air-conditioning failure, fire, theft, sabotage, overwriting disks or tapes by mistake, and mounting of a wrong tape by the operator.

Transaction and System Concepts

A transaction is an atomic unit of work that is either completed in its entirety or not done at all. A transaction passes through several states (Figure 21.4, similar to process states in operating systems).

Transaction states:

Active state (executing read, write operations)

Partially committed state (ended but waiting for system checks to determine success or failure)

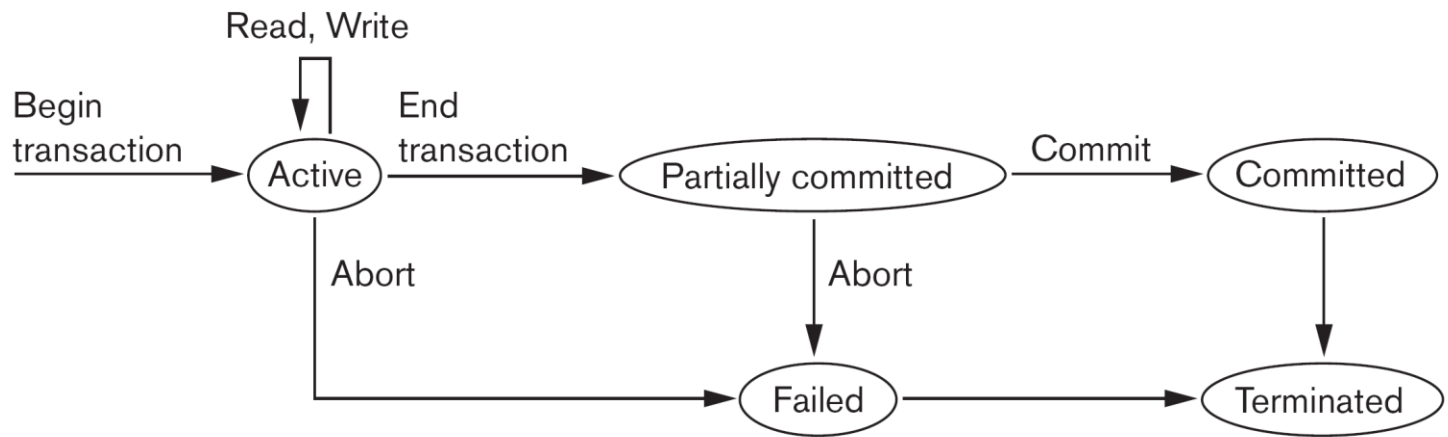
Committed state (transaction succeeded)

Failed state (transaction failed, must be rolled back)

Terminated State (transaction leaves system)

Figure 21.4

State transition diagram illustrating the states for transaction execution.



Transaction and System Concepts (cont.)

DBMS Recovery Manager needs system to keep track of the following operations (in the system log file):

- **begin_transaction:** Start of transaction execution.
- **read or write:** Read or write operations on the database items that are executed as part of a transaction.
- **end_transaction:** Specifies end of read and write transaction operations have ended. System may still have to check whether the changes (writes) introduced by transaction can be permanently applied to the database (commit transaction); or whether the transaction has to be rolled back (abort transaction) because it violates concurrency control or for some other reason.

Transaction and System Concepts (cont.)

Recovery manager keeps track of the following operations (cont.):

commit_transaction: Signals successful end of transaction; any changes (writes) executed by transaction can be safely committed to the database and will not be undone.

abort_transaction (or rollback): Signals transaction has ended unsuccessfully; any changes or effects that the transaction may have applied to the database must be undone.

Transaction and System Concepts (cont.)



System operations used during recovery

- **undo(X)**: Similar to rollback except that it applies to a single write operation rather than to a whole transaction.
- **redo(X)**: This specifies that a write operation of a committed transaction must be redone to ensure that it has been applied permanently to the database on disk.

Transaction and System Concepts (cont.)

The System Log File

- Is an append-only file to keep track of all operations of all transactions in the order in which they occurred. This information is needed during recovery from failures
- Log is kept on disk - not affected except for disk or catastrophic failure
- As with other disk files, a log main memory buffer is kept for holding the records being appended until the whole buffer is appended to the end of the log file on disk
- Log is periodically backed up to archival storage (tape) to guard against catastrophic failures

Transaction and System Concepts (cont.)

Types of records (entries) in log file:

- [start_transaction,T]: Records that transaction T has started execution.
- [write_item,T,X,old_value,new_value]: T has changed the value of item X from old_value to new_value.
- [read_item,T,X]: T has read the value of item X (not needed in many cases).
- [end_transaction,T]: T has ended execution
- [commit,T]: T has completed successfully, and committed.
- [abort,T]: T has been aborted.

Transaction and System Concepts (cont.)

The System Log (cont.):

- protocols for recovery that avoid cascading rollbacks do not require that read operations be written to the system log; most recovery protocols fall in this category.
- strict protocols require simpler write entries that do not include new value.

Transaction and System Concepts (cont.)



Commit Point of a Transaction:

A transaction T reaches its commit point when all its operations that access the database have been executed successfully and the effect of all the transaction operations on the database has been recorded in the log file (on disk). The transaction is then said to be committed.

Transaction and System Concepts (cont.)

Commit Point of a Transaction (cont.):

- **Log file buffers:** Like database files on disk, whole disk blocks must be read or written to main memory buffers.
- For **log file**, the last disk block (or blocks) of the file will be in main memory buffers to easily append log entries at end of file.
- **Force writing the log buffer:** before a transaction reaches its commit point, any main memory buffers of the log that have not been written to disk yet must be copied to disk.
- Called force-writing the log buffers before committing a transaction.
- Needed to ensure that any write operations by the transaction are recorded in the log file on disk before the transaction commits

Desirable Properties of Transactions

ACID properties:

Atomicity

Consistency

Isolation

Durability

- **Atomicity:** A transaction is an atomic unit of processing; it is either performed in its entirety or not performed at all.
- **Consistency preservation:** A correct execution of the transaction must take the database from one consistent state to another.

Desirable Properties of Transactions (cont.)

ACID properties (cont.):

- **Isolation:** Even though transactions are executing concurrently, they should appear to be executed in isolation – that is, their final effect should be as if each transaction was executed in isolation from start to finish.
- **Durability or permanency:** Once a transaction is committed, its changes (writes) applied to the database must never be lost because of subsequent failure.

Desirable Properties of Transactions (cont.)

- **Atomicity:** Enforced by the recovery protocol.
- **Consistency preservation:** Specifies that each transaction does a correct action on the database on its own. Application programmers and DBMS constraint enforcement are responsible for this.
- **Isolation:** Responsibility of the concurrency control protocol.
- **Durability or permanency:** Enforced by the recovery protocol.

Purpose of Concurrency Control

- To ensure that the Isolation Property is maintained while allowing transactions to execute concurrently (outcome of concurrent transactions *should appear as though they were executed in isolation*).
- To preserve database consistency by ensuring that the schedules of executing transactions are serializable.
- To resolve read-write and write-write conflicts among transactions.

Concurrency Control Protocols (CCPs)

- A CCP is a set of rules enforced by the DBMS to ensure serializable schedules
- Also known as CCMs (Concurrency Control Methods)
- Main protocol known as 2PL (2-phase locking), which is based on *locking the data items*
- Other protocols use different techniques
- We first cover 2PL in some detail, then give an overview of other techniques

2PL Concurrency Control Protocol

Based on each transaction securing a **lock** on a data item before using it:

Locking enforces **mutual exclusion** when accessing a data item – simplest kind of lock is a **binary lock**, which secures permission to Read or Write a data item for a transaction.

Example: If T1 requests Lock(X) operation, the system grants the lock *unless item X is already locked by another transaction*. If request is granted, data item X is locked on behalf of the requesting transaction T1.

Unlocking operation removes the lock.

Example: If T1 issues Unlock (X), data item X is made available to all other transactions.

2PL Concurrency Control Protocol



System maintains **lock table** to keep track of which items are locked by which transactions

Lock(X) and Unlock (X) are hence **system calls**

Transactions that request a lock but do not have it granted can be placed on a **waiting queue** for the item

Transaction T must unlock any items it had locked before T terminates

Next slides gives overview of lock and unlock operations

lock_item(X):

B: if $\text{LOCK}(X) = 0$ (* item is unlocked *)
 then $\text{LOCK}(X) \leftarrow 1$ (* lock the item *)
 else
 begin
 wait (until $\text{LOCK}(X) = 0$
 and the lock manager wakes up the transaction);
 go to **B**
 end;

unlock_item(X):

$\text{LOCK}(X) \leftarrow 0$; (* unlock the item *)
 if any transactions are waiting
 then wakeup one of the waiting transactions;

Figure 22.1

Lock and unlock operations for binary locks.

2PL Concurrency Control (cont.)

Locking for database items:

For database purposes, *binary locks are not sufficient*:

- Two locks modes are needed (a) **shared lock** (read lock) and (b) **exclusive lock** (write lock).

Shared mode: Read lock (X). Several transactions can hold shared lock on X (because read operations are not conflicting).

Exclusive mode: Write lock (X). Only one write lock on X can exist at any time on an item X. (No read or write locks on X by other transactions can exist).

Conflict matrix

	Read	Write
Read	Y	N
Write	N	N

2PL Concurrency Control (cont.)

Three operations are now needed:

- `read_lock(X)`: transaction T requests a read (shared) lock on item X
- `write_lock(X)`: transaction T requests a write (exclusive) lock on item X
- `unlock(X)`: transaction T unlocks an item that it holds a lock on (shared or exclusive)

Transaction can be blocked (forced to wait) if the item is held by other transactions **in conflicting lock mode**

Conflicts are write-write or read-write (read-read is not conflicting)

Next slide gives outline of these three operations

```

read_lock(X):
B:  if LOCK(X) = "unlocked"
      then begin LOCK(X) ← "read-locked";
           no_of_reads(X) ← 1
           end
      else if LOCK(X) = "read-locked"
           then no_of_reads(X) ← no_of_reads(X) + 1
      else begin
           wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
           go to B
           end;

write_lock(X):
B:  if LOCK(X) = "unlocked"
      then LOCK(X) ← "write-locked"
      else begin
           wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
           go to B
           end;

unlock (X):
      if LOCK(X) = "write-locked"
          then begin LOCK(X) ← "unlocked";
               wakeup one of the waiting transactions, if any
               end
      else if LOCK(X) = "read-locked"
          then begin
               no_of_reads(X) ← no_of_reads(X) - 1;
               if no_of_reads(X) = 0
                   then begin LOCK(X) = "unlocked";
                        wakeup one of the waiting transactions, if any
                        end
               end;

```

Figure 22.2
Locking and unlocking
operations for two-
mode (read-write or
shared-exclusive)
locks.

2PL Concurrency Control (cont.)

Two-Phase Locking Techniques: Essential components

Lock Manager: Subsystem of DBMS that manages locks on data items.

Lock table: Lock manager uses it to store information about *locked data items*, such as: data item id, transaction id, lock mode, list of waiting transaction ids, etc. One simple way to implement a lock table is through linked list (shown). Alternatively, a **hash table** with item id as hash key can be used.

Transaction ID	Data item id	lock mode	Ptr to next data item
T1	X1	Read	Next

2PL Concurrency Control (cont.)

Rules for locking:

- Transaction must request appropriate lock on a data item X before it reads or writes X .
- If T holds a write (exclusive) lock on X , it can both read and write X .
- If T holds a read lock on X , it can only read X .
- T must unlock all items that it holds before terminating
- (also T cannot unlock X unless it holds a lock on X).

2PL Concurrency Control (cont.)

Lock conversion

Lock upgrade: existing read lock to write lock

if T_i holds a read-lock on X , and no other T_j holds a read-lock on X ($i \neq j$), then it is possible to convert (**upgrade**) read-lock(X) to write-lock(X)

else

force T_i to wait until all other transactions T_j that hold read locks on X release their locks

Lock downgrade: existing write lock to read lock

if T_i holds a write-lock on X

(*this implies that no other transaction can have any lock on X^*)

then it is possible to convert (**downgrade**) write-lock(X) to read-lock(X)

2PL Concurrency Control (cont.)

Two-Phase Locking Rule:

Each transaction should have **two phases**: (a) Locking (Growing) phase, and (b) Unlocking (Shrinking) Phase.

Locking (Growing) Phase: A transaction applies locks (read or write) on desired data items one at a time. Can also try to upgrade a lock.

Unlocking (Shrinking) Phase: A transaction unlocks its locked data items one at a time. Can also downgrade a lock.

Requirement: For a transaction these two phases must be mutually exclusively, that is, during locking phase no unlocking or downgrading of locks can occur, and during unlocking phase no new locking or upgrading operations are allowed.

2PL Concurrency Control (cont.)

Basic Two Phase Locking:

When transaction starts executing, it is in the **locking phase**, and it can request locks on new items or upgrade locks. A transaction may be blocked (forced to wait) if a lock request is not granted. (This may lead to several transactions being in a *state of deadlock* – see later)

Once the transaction unlocks an item (or downgrades a lock), it starts its **shrinking phase** and can no longer upgrade locks or request new locks.

The combination of locking rules and 2-phase rule ensures serializable schedules

Theorem: If every *transaction* in a schedule follows the 2PL rules, the schedule must be serializable. (Proof is by contradiction.)

2PL Concurrency Control (cont.)

Some Examples:

Rules of locking alone do not enforce serializability – Figure 22.3(c) shows a schedule that follows locking rules but *is not serializable*

Figure 22.4 shows how the transactions in Figure 22.3(a) can be modified to follow the two-phase rule (by delaying the first unlock operation till all locking is completed).

The schedule in 22.3(c) would not be allowed if the transactions are changed as in 22.4 – a state of deadlock would occur instead because T2 would try to lock Y (which is locked by T1 in conflicting mode) and forced to wait, then T1 would try to lock X (which is locked by T2 in conflicting mode) and forced to wait – neither transaction can continue to unlock the item they hold as they are both blocked (waiting) – see Figure 22.5

T_1	T_2
read_lock(Y); read_item(Y); unlock(Y); write_lock(X); read_item(X); $X := X + Y$; write_item(X); unlock(X);	read_lock(X); read_item(X); unlock(X); write_lock(Y); read_item(Y); $Y := X + Y$; write_item(Y); unlock(Y);

Result serial schedule T_1
followed by T_2 : $X=50, Y=80$

Result of serial schedule T_2
followed by T_1 : $X=70, Y=50$

	T_1	T_2
Time ↓	read_lock(Y); read_item(Y); unlock(Y);	read_lock(X); read_item(X); unlock(X); write_lock(Y); read_item(Y); $Y := X + Y$; write_item(Y); unlock(Y);
	write_lock(X); read_item(X); $X := X + Y$; write_item(X); unlock(X);	

Result of schedule S :
 $X=50, Y=50$
 (nonserializable)

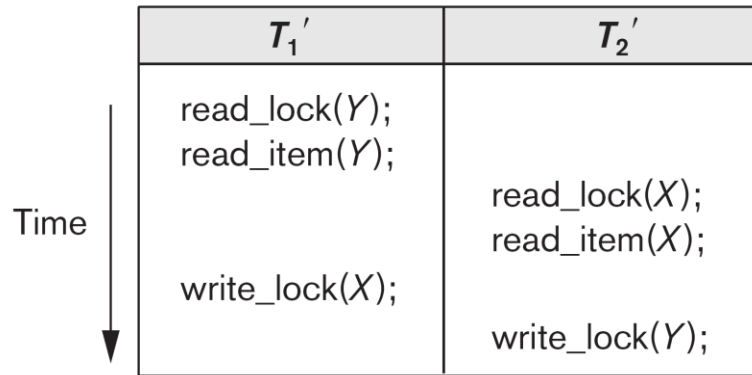
Transactions that do not obey two-phase locking. (a) Two transactions T_1 and T_2 . (b) Results of possible serial schedules of T_1 and T_2 . (c) A nonserializable schedule S that uses locks.

Figure 22.4

Transactions T_1' and T_2' , which are the same as T_1 and T_2 in Figure 22.3, but follow the two-phase locking protocol. Note that they can produce a deadlock.

T_1'	T_2'
read_lock(Y); read_item(Y); write_lock(X); unlock(Y) read_item(X); $X := X + Y$; write_item(X); unlock(X);	read_lock(X); read_item(X); write_lock(Y); unlock(X) read_item(Y); $Y := X + Y$; write_item(Y); unlock(Y);

(a)



(b)

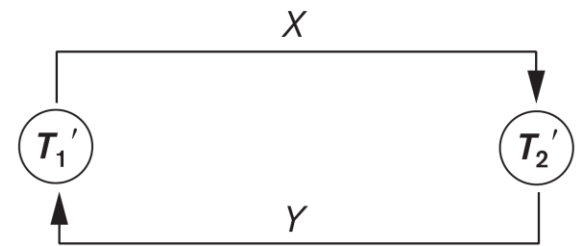


Figure 22.5

Illustrating the deadlock problem. (a) A partial schedule of T_1' and T_2' that is in a state of deadlock. (b) A wait-for graph for the partial schedule in (a).

2PL Concurrency Control (cont.)

Other Variations of Two-Phase Locking:

Conservative 2PL: A transaction must *lock all its items before starting execution*. If any item it needs is not available, it locks no items and tries again later. Conservative 2PL *has no deadlocks* since no lock requests are issued once transaction execution starts.

Strict 2PL: All items that are *writelocked* by a transaction are not released (unlocked) until *after the transaction commits*. This is the most commonly used two-phase locking algorithm, and ensures strict schedules (for recoverability).

Rigorous 2PL: All items that are *writelocked or readlocked* by a transaction are not released (unlocked) until *after the transaction commits*. Also guarantees strict schedules.

2PL Concurrency Control (cont.)

Deadlock Example (see Figure 22.5)

T1'

read_lock (Y);

read_item (Y);

write_lock (X);

(waits for X)

T2'

read_lock (X);

read_item (Y);

write_lock (Y);

(waits for Y)

T1' and T2' did follow two-phase policy but they are deadlock

Deadlock (T1' and T2')

2PL Concurrency Control (cont.)

Dealing with Deadlock

Deadlock prevention

System enforces additional rules (deadlock prevention protocol) to ensure deadlock do not occur – many protocols (see later for some examples)

Deadlock detection and resolution

System checks for a state of deadlock – if a deadlock exists, one of the transactions involved in the deadlock is aborted

2PL Concurrency Control (cont.)

1. Deadlock detection and resolution

In this approach, deadlocks are allowed to happen. The system maintains a *wait-for graph* for detecting cycles. If a cycle exists, then one transaction involved in the cycle is selected (victim) and rolled-back (aborted).

A **wait-for graph** contains a node for each transaction. When a transaction (say T_i) is blocked because it requests an item X held by another transaction T_j in conflicting mode, a directed edge is created from T_i to T_j (T_i is waiting on T_j to unlock the item). The system checks for cycles; if a cycle exists, a state of deadlock is detected (see Figure 22.5).

2PL Concurrency Control (cont.)

2. Deadlock prevention

There are several protocols. Some of them are:

1. Conservative 2PL, as we discussed earlier.
2. No-waiting protocol: A transaction never waits; if T_i requests an item that is held by T_j in conflicting mode, T_i is aborted. Can result in *needless transaction aborts* because deadlock might have never occurred
3. Cautious waiting protocol: If T_i requests an item that is held by T_j in conflicting mode, the system checks the status of T_j ; if T_j is *not blocked*, then T_i waits – if T_j is *blocked*, then T_i aborts. Reduces the number of needlessly aborted transactions

2PL Concurrency Control (cont.)

2. Deadlock prevention (cont.)

4. Wound-wait and wait-die: Both use transaction timestamp $TS(T)$, which is a monotonically increasing unique id given to each transaction based on their starting time

$TS(T1) < TS(T2)$ means that $T1$ started before $T2$ ($T1$ older than $T2$)

(Can also say $T2$ younger than $T1$)

Wait-die:

If T_i requests an item X that is held by T_j in conflicting mode, then

if $TS(T_i) < TS(T_j)$ then T_i waits (on a younger transaction T_j)

else T_i dies (if T_i is younger than T_j , it aborts)

[In wait-die, transactions only wait on younger transactions that started later, so no cycle ever occurs in wait-for graph – if transaction requesting lock is younger than that holding the lock, requesting transaction aborts (dies)]

2PL Concurrency Control (cont.)

2. Deadlock prevention (cont.)

4. Wound-wait and wait-die (cont.):

Wound-wait:

If T_i requests an item X that is held by T_j in conflicting mode, then

if $TS(T_i) < TS(T_j)$ then T_j is aborted (T_i wounds younger T_j)

else T_i waits (on an older transaction T_j)

[In wound-wait, transactions only wait on older *transactions* that started earlier, so no cycle ever occurs in wait-for graph – if transaction requesting lock is older than that holding the lock, transaction holding the lock is preemptively aborted]

Database Concurrency Control

Dealing With Starvation

Starvation occurs when a particular transaction consistently waits or gets restarted and never gets a chance to proceed further. Solution is to use a *fair priority-based scheme* that increases the priority for transaction the longer they wait.

Examples of Starvation:

1. In deadlock detection/resolution it is possible that the same transaction may consistently be selected as victim and rolled-back.
2. In conservative 2PL, a transaction may never get started because all the items needed are never available at the same time.
3. In Wound-Wait scheme a younger transaction may always be wounded (aborted) by a long running older transaction which may create starvation.

Other Concurrency Control Methods

Some CCMs other than 2PL:

- Timestamp Ordering (TO) CCM
- Optimistic (Validation-Based) CCMs
- Multi-version CCMs:
 - Multiversion 2PL with Certify Locks
 - Multiversion TO

Purpose of Database Recovery

- To bring the database into a consistent state after a failure occurs.
- To ensure the transaction properties of **Atomicity** (a transaction must be done in its entirety; otherwise, it has to be rolled back) and **Durability** (a committed transaction cannot be canceled and all its updates must be applied permanently to the database).
- After a failure, the **DBMS recovery manager** is responsible for bringing the system into a consistent state before transactions can resume.

Types of Failure

- Transaction failure: Transactions may fail because of errors, incorrect input, deadlock, incorrect synchronization, etc.
- System failure: System may fail because of application error, operating system fault, RAM failure, etc.
- Media failure: Disk head crash, power disruption, etc.

The Log File

- Holds the information that is necessary for the recovery process
- Records all relevant operations in the order in which they occur (looks like a *schedule of transactions*, see Chapter 21)
- Is an append-only file.
- Holds various types of log records (or log entries).

Log File Entries (from Chapter 21)

Types of records (entries) in log file:

- [start_transaction,T]: Records that transaction T has started execution.
- [write_item,T,X,old_value,new_value]: T has changed the value of item X from old_value to new_value.
- [read_item,T,X]: T has read the value of item X (not needed in many cases).
- [end_transaction,T]: T has ended execution
- [commit,T]: T has completed successfully, and committed.
- [abort,T]: T has been aborted.

The Log File (cont.)

For **write_item** log entry, *old value* of item before modification (**BFIM** - BeFore Image) and the *new value* after modification (**AFIM** – AFTer Image) are stored. BFIM needed for UNDO, AFIM needed for REDO. A sample log is given below. **Back P** and **Next P** point to the previous and next log records of the same transaction.

T ID	Back P	Next P	Operation	Data item	BFIM	AFIM
T1	0	1	Begin			
T1	1	4	Write	X	X = 100	X = 200
T2	0	8	Begin			
T1	2	5	W	Y	Y = 50	Y = 100
T1	4	7	R	M	M = 200	M = 200
T3	0	9	R	N	N = 400	N = 400
T1	5	nil	End			

Database Cache

Database Cache: A set of *main memory* buffers; each buffer typically holds contents of one disk block. Stores the disk blocks that contain the data items being read and written by the database transactions.

Data Item Address: (disk block address, offset, size in bytes).

Cache Table: Table of entries of the form (buffer addr, disk block addr, modified bit, pin/unpin bit, ...) to indicate which disk blocks are currently in the cache buffers.

Database Cache (cont.)

Data items to be modified are first copied into database cache by the Cache Manager (CM) and after modification they are flushed (written) back to the disk. The flushing is controlled by **Modified** and **Pin-Unpin** bits.

Pin-Unpin: If a buffer is pinned, it cannot be written back to disk until it is unpinned.

Modified: Indicates that one or more data items in the buffer have been changed.

Data Update

- **Immediate Update:** A data item modified in cache can be written back to disk *before the transaction commits*.
- **Deferred Update:** A modified data item in the cache cannot be written back to disk till *after the transaction commits* (buffer is **pinned**).
- **Shadow update:** The modified version of a data item does not overwrite its disk copy but is written at a separate disk location (new version).
- **In-place update:** The disk version of the data item is overwritten by the cache version.

UNDO and REDO Recovery Actions

To maintain atomicity and durability, some transaction's may have their operations **redone** or **undone** during recovery. UNDO (roll-back) is needed for transactions that are not committed yet. REDO (roll-forward) is needed for committed transactions whose writes may have not yet been flushed from cache to disk.

Undo: Restore all BFIMs from log to database on disk. UNDO proceeds backward in log (from most recent to oldest UNDO).

Redo: Restore all AFIMs from log to database on disk. REDO proceeds forward in log (from oldest to most recent REDO).

Write-ahead Logging Protocol

The information needed for recovery must be written to the log file on disk before changes are made to the database on disk.

Write-Ahead Logging (WAL) protocol consists of two rules:

For Undo: Before a data item's AFIM is flushed to the database on disk (overwriting the BFIM) its BFIM must be written to the log and the log must be saved to disk.

For Redo: Before a transaction executes its commit operation, all its AFIMs must be written to the log and the log must be saved on a stable store.

Deferred Update (NO-UNDO/REDO) Recovery Protocol

System must impose **NO-STEAL** rule. Recovery subsystem analyzes the log, and creates two lists:

Active Transaction list: All active (uncommitted) transaction ids are entered in this list.

Committed Transaction list: Transactions committed after the last checkpoint are entered in this table.

During recovery, transactions in **commit** list are **redone**; transactions in **active** list are *ignored* (because of NO-STEAL rule, none of their writes have been applied to the database on disk). Some transactions may be **redone** twice; this does not create inconsistency because **REDO** is “**idempotent**”, that is, one REDO for an AFIM is equivalent to multiple REDO for the same AFIM.

Deferred Update (NO-UNDO/REDO) Recovery Protocol (cont.)



Advantage: Only **REDO** is needed during recovery.

Disadvantage: Many buffers may be pinned while waiting for transactions that updated them to commit, so system may run out of cache buffers when requests are made by new transactions.

UNDO/NO-REDO Recovery Protocol

In this method, **FORCE** rule is imposed by system (AFIMs of a transaction are flushed to the database on disk under Write Ahead Logging *before the transaction commits*).

Transactions in active list are **undone**; transactions in committed list are ignored (because based on FORCE rule, all their changes are already written to the database on disk).

Advantage: During recovery, only **UNDO** is needed.

Disadvantages: 1. Commit of a transaction is delayed until all its changes are force-written to disk. 2. Some buffers may be written to disk multiple times if they are updated by several transactions.

UNDO/REDO Recovery Protocol

Recovery can require both UNDO of some transactions and REDO of other transactions (Corresponds to STEAL/NO-FORCE). Used most often in practice because of disadvantages of the other two methods. To minimize REDO, checkpointing is used. The recovery performs:

1. **Undo** of a transaction if it is in the active transaction list.
2. **Redo** of a transaction if it is in the list of transactions that committed since the last checkpoint.

Why NoSQL?

- **Flexibility:** In general, NoSQL databases have flexible structures that allow for rapid and much more iterative improvement. Because of its flexible data format, NoSQL databases are suitable for semi-structured and unstructured information.
- **Scalability:** NoSQL databases are usually designed to expand by employing dispersed hardware groups rather than expanding higher by introducing costly and resilient servers. Like a service software, several cloud services perform these activities behind the lines.
- **High-performance:** NoSQL databases are designed for specific data structures and associated metadata, resulting in faster responsiveness than attempting to provide equivalent functionality using relational databases.
- **Enhanced functionality:** NoSQL databases offer purpose-built data types which offer distinct data models and also a highly functioning Interfaces.

Characteristics of NoSQL

- it's more than rows in tables—NoSQL systems store and retrieve data from many formats: key-value stores, graph databases, column-family (Bigtable) stores, document stores, and even rows in tables.
- It's free of joins—NoSQL systems allow you to extract your data using simple interfaces without joins.
- It's schema-free—NoSQL systems allow you to drag-and-drop your data into a folder and then query it without creating an entity-relational model.
- It works on many processors—NoSQL systems allow you to store your database on multiple processors and maintain high-speed performance.
- It uses shared-nothing commodity computers—Most (but not all) NoSQL systems leverage low-cost commodity processors that have separate RAM and disk.
- It supports linear scalability—When you add more processors, you get a consistent increase in performance.
- It's innovative—NoSQL offers options to a single way of storing, retrieving, and manipulating data. NoSQL supporters (also known as NoSQLers) have an inclusive attitude about NoSQL and recognize SQL solutions as viable options. To the NoSQL community, NoSQL means “Not only SQL.”

Use of NoSQL in industry

- **Session Store:** Managing session data using relational database is very difficult, especially in case where applications are grown very much.
- **User Profile Store:** To enable online transactions, user preferences, authentication of user and more, it is required to store the user profile by web and mobile application.
- **Content and Metadata Store:** Many companies like publication houses require a place where they can store large amount of data, which include articles, digital content and e-books, in order to merge various tools for learning in single platform
- **Mobile Applications:** Since the smartphone users are increasing very rapidly, mobile applications face problems related to growth and volume.
- **Third-Party Data Aggregation:** Frequently a business require to access data produced by third party. For instance, a consumer packaged goods company may require to get sales data from stores as well as shopper's purchase history.

Use of NoSQL in industry

- **Internet of Things:** Today, billions of devices are connected to internet, such as smartphones, tablets, home appliances, systems installed in hospitals, cars and warehouses. For such devices large volume and variety of data is generated and keep on generating.
- **E-Commerce**
 - ▣ E-commerce companies use NoSQL for store huge volume of data and large amount of request from user.
- **Social Gaming** Data-intensive applications such as social games which can grow users to millions. Such a growth in number of users as well as amount of data requires a database system which can store such data and can be scaled to incorporate number of growing users NOSQL is suitable for such applications.
- **Ad Targeting** Displaying ads or offers on the current web page is a decision with direct income To determine what group of users to target, on web page where to display ads, the platforms gathers behavioral and demographic characteristics of users.

SQL VS NoSQL

Index	SQL	NoSQL
1)	Databases are categorized as Relational Database Management System (RDBMS).	NoSQL databases are categorized as Non-relational or distributed database system.
2)	SQL databases have fixed or static or predefined schema.	NoSQL databases have dynamic schema.
3)	SQL databases display data in form of tables so it is known as table-based database.	NoSQL databases display data as collection of key-value pair, documents, graph databases or wide-column stores.
4)	SQL databases are vertically scalable.	NoSQL databases are horizontally scalable.
5)	SQL databases use a powerful language "Structured Query Language" to define and manipulate the data.	In NoSQL databases, collection of documents are used to query the data. It is also called unstructured query language. It varies from database to database.
6)	SQL databases are best suited for complex queries.	NoSQL databases are not so good for complex queries because these are not as powerful as SQL queries.
7)	SQL databases are not best suited for hierarchical data storage.	NoSQL databases are best suited for hierarchical data storage.
8)	MySQL, Oracle, Sqlite, PostgreSQL and MS-SQL etc. are the example of SQL database.	MongoDB, BigTable, Redis, RavenDB, Cassandra, Hbase, Neo4j, CouchDB etc. are the example of nosql database

MongoDB

- ❑ MongoDB is a document database with the scalability and flexibility that you want with the querying and indexing that you need.
- ❑ The MongoDB database has a flexible data model that enables you to store unstructured data, and it provides full indexing support, and replication with rich and intuitive APIs.

why is mongodb

here are four major reasons why MongoDB is being deployed more often. They are:

1. Flexibility:

MongoDB uses documents that can contain sub-documents in complex hierarchies making it expressive and flexible. MongoDB can map objects from any programming language, ensuring easy implementation and maintenance.

2. Flexible Query Model:

The user can selectively index some parts of each document or a query based on regular expressions, ranges, or attribute values, and have as many properties per object as needed by the application layer.

3. Native Aggregation:

Native aggregation allows users to extract and transform data from the database. The data can either be loaded into a new format or exported to other data sources.

4. Schema-less model:

Applications get the power and responsibility to interpret different properties found in a collection's documents.